Space Empire

Roadmap & Organisation des développements

RÉPARTITION PAS OUF / MAIS BON DÉCOUPAGE

Priorisation des features — Répartition Thomas / Binôme

Mars 2026

1. Principes de la roadmap

1.1 Contraintes prises en compte

- Binôme débutant en Symfony/PHP — montée en compétence progressive obligatoire
- Projet long terme sans deadline — on privilégie la qualité et la solidité des bases
- Travail en parallèle maximisé — Thomas sur le backend complexe, binôme sur les domaines accessibles
- Pas de sur-architecture prématurée — on construit dans l'ordre de la dépendance technique

1.2 Répartition générale des responsabilités

Thomas	Binôme
• Architecture & infrastructure (Docker, Traefik, RabbitMQ, Mercure) • Logique métier complexe (combat, flotte, calculs univers) • Services Symfony (UniverseContext, CombatResolver...) • RabbitMQ — messages différés • Revue de code du binôme • Décisions d'architecture * Script Python de génération de carte	• Entités Doctrine & migrations (guidé par Thomas) • Fichiers YAML de catalogue (vaisseaux, bâtiments, techno...) • Templates Twig (pages, layouts, composants UI) • Controllers simples (lecture seule, affichage) • CSS / Stimulus (comportements UI) • ~~Script Python de génération de carte~~

> *Le binôme ne touche jamais à un service métier ou un MessageHandler sans relecture de Thomas. En revanche, il est autonome sur les entités, le YAML, Twig et le CSS une fois les patterns montrés.*
>
> *Thomas ne push pas dans que Caim n'a pas regarder ce qu'il a fait, pour pouvoir poser des questions*

2. Les 6 phases de développement

Phase	Objectif	Livrable
Phase 1	Socle technique — projet qui démarre, BDD connectée, authentification	Login fonctionnel
Phase 2	Univers & carte — import JSON, PlayerUniverse, sélection d'univers	Joueur dans un univers
Phase 3	Colonie — ressources, bâtiments, recherches, chantier	Planète jouable
Phase 4	Flottes & déplacements — envoi, calculs, cargo, retour	Flotte qui bouge
Phase 5	Combat & guildes — résolution, camps, logs, relations	Jeu complet MVP
Phase 6	Multi-univers avancé — modes de jeu, variantes, UI complète	Produit final

3. Phase 1 — Socle technique

Objectif : avoir un projet Symfony qui démarre, avec Docker, la BDD connectée, l'authentification par session et la structure de code en place. Rien de jouable encore — c'est la fondation sur laquelle tout le reste s'appuie.

	Thomas (bleu)	Binôme (vert)	Durée
Infra Docker	• Dockerfile PHP-FPM 8.3 • Docker Compose (nginx, mariadb, rabbitmq, mercure) • Traefik + certificats • Entrypoint.sh (migrations + import)	• Lire et comprendre le docker-compose • Tester le démarrage en local	**S1**
Structure projet	• Installer Symfony 7 + tous les bundles • Créer l'arborescence src/ (Service/, Manager/, Validator/DTO/, etc.) • Configurer security.yaml, messenger.yaml, mercure.yaml	• Installer les dépendances npm (Webpack Encore, Stimulus) • Créer le layout Twig de base (base.html.twig) • Mettre en place le CSS global	**S1**
Entité Player & auth	• Entity Player + migration • Formulaire de login/logout (Security) • Guard authenticator • Fixtures de test (1 joueur admin)	• Page de login Twig (formulaire, erreurs) • Page d'accueil après connexion (stub) • Controller HomeController (page vide connectée)	**S1-S2**

> *À la fin de la Phase 1 : docker compose up démarre tout, un joueur peut se connecter et voir une page d'accueil vide. Le binôme a compris l'architecture de base.*

4. Phase 2 — Univers & carte

Objectif : un joueur peut choisir un univers et y être rattaché. La carte (galaxies, systèmes, planètes) est importée depuis le JSON. L'UniverseContext est opérationnel.

	Thomas (bleu)	Binôme (vert)	Durée
Entités univers	• Entity Universe, PlayerUniverse • Entity Galaxy, SolarSystem, Planet • Migrations correspondantes • UniverseManager, PlanetManager	• YAML config des 2 premiers univers (omega-1.yaml, turbo-1.yaml) • Script Python universe_generator.py (génère le JSON de carte)	**S2**
Import & context	• UniverseImportCommand (app:universe:import) • UniverseImporter (Service/Universe/) • UniverseContext — getPlayerUniverse(), getMultiplier(), isFeatureEnabled() • EventListener redirect si univers null/fermé	• JSON de carte pour omega-1 (via script Python) • JSON de carte pour turbo-1 • Tester le script Python en local	**S2-S3**
Sélection univers	• UniverseController (liste + changement d'univers) • PlayerManager — createOrGetPlayerUniverse() • Logique current_player_universe sur Player	• Page de sélection des univers (Twig) • Carte de chaque univers (nom, description, statut, vitesse) • Redirection automatique si déconnecté de l'univers	**S3**

> *À la fin de la Phase 2 : le joueur voit la liste des univers, en choisit un, et est rattaché à un PlayerUniverse. La carte est en BDD. L'UniverseContext est prêt pour toutes les phases suivantes.*

5. Phase 3 — Colonie

Objectif : le joueur colonise une planète, voit ses ressources évoluer, construit des bâtiments, lance des recherches et commande des vaisseaux. Première mécanique asynchrone via RabbitMQ (fin de construction).

	Thomas (bleu)	Binôme (vert)	Durée
Catalogues YAML	• UniverseCatalog, ShipCatalog, BuildingCatalog, TechnologyCatalog, DefenseCatalog • ValueObjects — ShipDefinition, BuildingDefinition, etc. • RequirementChecker, ShipUnlockChecker	• Rédiger tous les fichiers YAML vaisseaux (14 slugs) • Rédiger tous les fichiers YAML bâtiments • Rédiger tous les fichiers YAML technologies	**S3-S4**
Ressources	• Entity PlanetResource + migration • ResourceService — lazy calculation + multiplicateurs univers • ResourceManager • ColonizationService (première planète du joueur)	• Page de la planète — affichage ressources en temps réel (Twig + Stimulus counter) • Barre de progression des ressources	**S4**
Bâtiments	• Entity PlanetBuilding + migration • BuildingService — vérif prérequis, déduction ressources • BuildingManager • Message BuildingUpgradeFinished + Handler	• Page des bâtiments — liste, niveau actuel, bouton upgrade • Affichage du timer de construction (Stimulus countdown) • Notification Mercure fin de construction (Stimulus)	**S4-S5**
Recherches	• Entity PlayerTechnology, ResearchQueue + migrations • ResearchService — contraintes (1 active), ResearchTimeCalculator • ResearchManager • Message ResearchFinished + Handler	• Page des recherches — arbre de technologies, prérequis visuels • Affichage du timer de recherche en cours	**S5**
Chantier spatial	• Entity ShipyardQueue + migration • ShipyardService, BuildTimeCalculator • ShipyardManager • Message ShipConstructionFinished + Handler	• Page du chantier — catalogue vaisseaux, prérequis, file de construction • Affichage de la queue avec timers	**S5-S6**

> *À la fin de la Phase 3 : un joueur peut coloniser une planète, voir ses ressources augmenter en temps réel, construire des bâtiments, chercher des technologies et commander des vaisseaux. Les timers RabbitMQ fonctionnent.*

6. Phase 4 — Flottes & déplacements

Objectif : le joueur peut envoyer une flotte vers une autre planète. Les calculs de distance, de carburant, de temps de trajet et les messages RabbitMQ d'arrivée sont opérationnels.

	Thomas (bleu)	Binôme (vert)	Durée
Entités flotte	• Entity Fleet, FleetShip + migrations • FleetManager • FleetRepository — requêtes filtrées par PlayerUniverse	• Page de mes flottes — liste, statut, destination, ETA • Carte interactive simplifiée (Stimulus + SVG ou canvas)	**S6**
Mouvement	• FleetMovementService — distance 3 segments, temps de trajet • CargoService — calcul hydrogène, validation soute • FleetService — envoi, retour, changement statut • Message FleetArrived + Handler	• Formulaire d'envoi de flotte (destination, composition, vitesse) • Affichage ETA et consommation carburant estimée	**S6-S7**
Fog of War	• Entity PlayerDiscovery + migration • FogOfWarService — découverte système à l'arrivée d'une sonde • Filtrage des réponses API selon découvertes	• Affichage différencié carte — systèmes connus vs inconnus • Icône sonde, animation de scan (Stimulus)	**S7**
Orbite & transport	• Gestion statut orbit — durée limitée par carburant • Transport de ressources — chargement/déchargement • Gestion du retour automatique	• Page détail d'une flotte en déplacement • Barre de progression du trajet (Stimulus countdown)	**S7**

> *À la fin de la Phase 4 : le jeu est déjà partiellement jouable en solo. On peut coloniser, développer une planète, envoyer des flottes, explorer la carte.*

7. Phase 5 — Combat & guildes

Objectif : le jeu devient multi-joueurs. Les flottes peuvent s'attaquer, les combats se résolvent avec logs, les guildes et alliances fonctionnent.

	Thomas (bleu)	Binôme (vert)	Durée
Relations joueurs	• Entity Guild, GuildMember, GuildRelation, AllianceContract + migrations • GuildManager, AllianceService, GuildRelationService, RelationResolver • GuildController	• Page de guilde — membres, statut, relations inter-guildes • Page des alliances — contrats actifs, demandes en attente	**S7-S8**
Système de combat	• Entity CombatEvent, CombatParticipant, CombatLog + migrations • CombatEventService — timer, arrivées, fenêtre de renfort • CampResolver — détermination des camps (1v1, coalition, FFA) • HitCalculator — formule de toucher, modificateurs • CombatResolver — rounds, destructions, butin • TechnologyBonusResolver • Message CombatResolve + Handler	• Page de rapport de combat — log des rounds, pertes, butin • Historique des combats d'une planète	**S8-S9**
Notifications	• CombatNotificationService — qui notifier, visibilité attaquant • FleetNotificationService • MercurePublisher — publication des événements SSE	• Controller Stimulus de notifications (toast, alertes, sons) • Bannière d'alerte attaque en cours (temps réel)	**S9**
Défenses	• Entity PlanetDefense + migration • DefenseService — construction, intégration au combat • DefenseCatalog	• YAML des défenses statiques • Page des défenses d'une planète	**S9**

> *À la fin de la Phase 5 : le jeu est un MVP complet et jouable. Plusieurs joueurs peuvent s'affronter dans un univers, former des guildes, se battre et explorer.*

8. Phase 6 — Multi-univers avancé & polish

Objectif : exploiter pleinement le système multi-univers. Ajout des modes de jeu spéciaux, fermeture automatique des univers temporaires, UI complète et polish général.

	Thomas (bleu)	Binôme (vert)	Durée
Modes de jeu	• Feature flags — planet_conquest, guilds_enabled dans UniverseContext • ColonizationService — vol de planète si feature activée • Univers temporaires — Message UniverseClose + Handler	• YAML des univers spéciaux (conquest.yaml, speed_x10.yaml...) • Page de sélection enrichie (description du mode, règles)	**S9-S10**
Classements	• Requêtes de classement par univers (points, flottes, planètes) • API endpoints classement	• Page classement par univers — tableau, podium • Statistiques personnelles par univers	**S10**
UI & polish	• Optimisation des requêtes Doctrine (N+1, index manquants) • Mise en place du cache Symfony pour les catalogues	• Refonte UI complète — design cohérent, responsive • Animations Stimulus (transitions, effets combat) • Carte interactive avancée • Dark mode	**S10+**
Qualité	• Tests unitaires Service/ (PHPUnit) • Tests fonctionnels Controller/ • CI/CD GitLab — lint, tests automatiques	• Tests des pages Twig (accessibilité, rendu) • Documentation utilisateur	**En continu**

9. Planning et priorisation

9.1 Ordre de priorité des features

#	Feature	Pourquoi cette priorité	Phase
1	Socle Docker + Symfony	Tout le reste en dépend	Phase 1
2	Authentification (session)	Nécessaire avant toute page	Phase 1
3	Univers + PlayerUniverse	Toutes les entités en dépendent	Phase 2
4	Import de carte (JSON → BDD)	Nécessaire avant colonie	Phase 2
5	UniverseContext	Injecté dans tous les services	Phase 2
6	Catalogues YAML	Prérequis du chantier et des bâtiments	Phase 3
7	Ressources (lazy calc)	Mécanique centrale du jeu	Phase 3
8	Bâtiments + RabbitMQ	Premier timer asynchrone	Phase 3
9	Recherches technologiques	Prérequis des vaisseaux avancés	Phase 3
10	Chantier spatial	Prérequis des flottes	Phase 3
11	Flottes + déplacements	Prérequis du combat	Phase 4
12	Fog of War	Explorations et sondes	Phase 4
13	Guildes + alliances	Prérequis du combat multi	Phase 5
14	Système de combat	Feature centrale multi-joueurs	Phase 5
15	Notifications Mercure	UX temps réel	Phase 5
16	Modes de jeu (univers)	Différenciation des univers	Phase 6
17	Classements	Engagement joueur long terme	Phase 6
18	Tests & CI/CD	Qualité, en continu dès Phase 3	Continu

9.2 Ce qui ne doit PAS être fait trop tôt

- Optimisation des performances — seulement quand le jeu est jouable et qu'un problème réel est observé
- Redis — pas avant d'avoir des joueurs réels et des mesures de perf
- Turbo (navigation SPA) — ajoute de la complexité au cycle de vie Stimulus, post-MVP uniquement
- Classements et statistiques avancées — inutile sans une base de joueurs
- Modes de jeu spéciaux — d'abord un univers standard qui fonctionne bien

9.3 Points de synchronisation obligatoires

Avant de commencer chaque phase, Thomas et le binôme doivent faire un point ensemble :

Avant la phase	Point de synchronisation
Phase 2	Thomas explique Universe/PlayerUniverse et montre un exemple d'entité + migration
Phase 3	Thomas montre le pattern Service → Manager → Repository sur ResourceService. Le binôme reproduit pour BuildingService sous supervision.
Phase 4	Thomas explique RabbitMQ et les DelayStamp. Le binôme lit le code du Handler avant de toucher aux pages Twig.
Phase 5	Thomas explique le CombatResolver en détail. Le binôme n'intervient que sur le frontend.
Phase 6	Bilan MVP — que manque-t-il ? Que retravailler ? Priorisation collective.

Roadmap v1 — Mars 2026